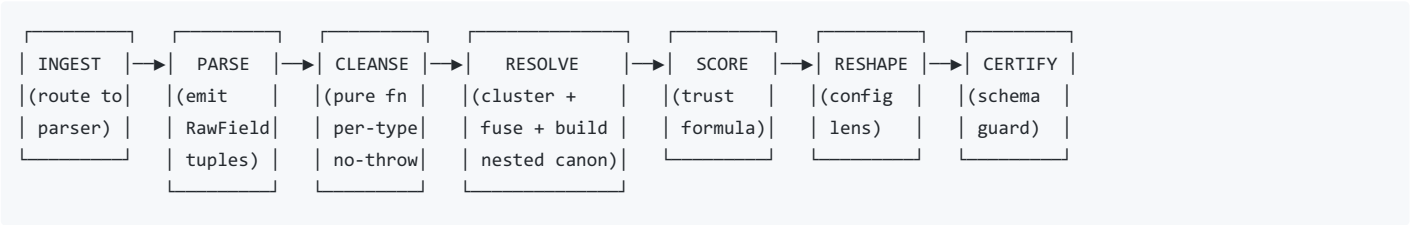


Multi-Source Candidate Data Transformer

Problem

Candidate data lives across disconnected systems (CSV, ATS JSON, GitHub, resumes, notes) with conflicting values, different schemas, and varying reliability. We need one canonical profile per candidate — normalized, deduplicated, confidence-scored, and projectable into any output shape via runtime config.

Architecture — 7-Stage Pipeline



Stage	What It Does	Design Choice
Ingest	Routes files to parsers by extension + JSON key inspection	Content-aware: distinguishes ATS/GitHub/LinkedIn JSON by structure, not filename
Parse	Extractors emit <code>RawField(field, value, source, method)</code> tuples	Flat fields only — extractors never build nested objects. Adding a new source = one file
Cleanse	Pure normalizers: phone→E.164, country→ISO-3166, skill→taxonomy	Returns <code>None</code> on bad input — never throws , never guesses
Resolve	Entity resolution + flat-to-nested canonical build	Single adapter layer : email→phone→fuzzy cascade merges duplicates, builds nested schema
Score	Per-field confidence via documented formula	Corroboration boosts score, conflicts penalize it
Reshape	Config-driven projection with path remapping	Config is a read-only lens — same data, different output shapes, zero code changes
Certify	JSON-schema validation of canonical + projected output	Two-layer: Pydantic (internal) + jsonschema (output conformance)

Key Design Decisions

- Canonical record is immutable; config is a read-only lens.** The `CandidateProfile` stores every field we have. The runtime config never mutates it — it only selects, remaps, and reshapes for output. This means adding new output formats (a recruiter dashboard, an API response, an internal report) never risks data corruption. The canonical record is the single source of truth.
- Extractors emit flat fields; Resolve is the single adapter layer.** Every extractor (CSV, ATS, GitHub, Resume, Notes) emits simple flat key-value tuples: `email=alice@example.com, skill=Python, current_title=Senior Engineer`. The **Resolve** stage is the only place that transforms flat → nested (`email → emails[], current_title → headline, skills → [{name, confidence, sources[]}]`). This decoupling means adding a new data source is just one new extractor file — you don't touch the schema or merge logic.
- Content-aware ingestion, not filename hardcoding.** The Ingest stage inspects JSON key structure (not just file extensions) to distinguish ATS exports, GitHub user lists, and LinkedIn fixtures — making the pipeline resilient to arbitrary file naming.

Canonical Schema & Normalization

The `CandidateProfile` (Pydantic v2) uses nested objects: location {city, region, country}, links {linkedin, github, portfolio, other[]}, skills [{name, confidence, sources[]}], experience [{company, title, start, end, summary}], education [{institution, degree, field, end_year}], provenance [{field, source, method}], and `overall_confidence`.

6 normalizers (all pure functions — no side effects, return `None` on unparseable input):

- Phone:** (415) 555-0101 → +14155550101 (E.164 via `phonenumbers`)
- Country:** United States / USA → US (ISO-3166 via `pycountry`)
- Date:** Jan 2024 → 2024-01 (YYYY-MM via `python-dateutil`)
- Skill:** js / k8s / postgres → JavaScript / Kubernetes / PostgreSQL (rapidfuzz, 85+ entry alias map)

- **Email:** `Alice@Example.COM` → `alice@example.com`
- **Name:** `alice chen` → `Alice Chen` (title case)

Merge & Conflict Resolution

Entity resolution cascade (strongest signal first): Email (exact) → Phone (E.164 normalized) → Fuzzy name+company (rapidfuzz ≥85, requires both to match).

Conflict policy: Source priority order `csv > ats_json > github > linkedin > resume > notes` (structured > unstructured). Tiebreaker: most recent timestamp.

Exception: skills, emails, and phones are **union-merged**, not pick-winner.

Confidence Scoring

$$\text{field_confidence} = \text{base_weight}(\text{source}) \times \text{method_modifier} + \text{corroboration_bonus} - \text{conflict_penalty}$$

- **Source weights:** `csv=0.80, ats_json=0.85, github=0.70, linkedin=0.90, resume=0.60, notes=0.40`
- **Method modifiers:** `structured_parse=1.0, api=0.95, regex=0.80, heuristic=0.60`
- **Corroboration:** +0.10 per additional agreeing source (capped at +0.30)
- **Conflict penalty:** -0.15 when sources disagree
- **Overall:** weighted average where identity fields (`emails=2.0, name=1.5`) weigh more than supplementary (`links=0.4`)

Projection Engine

- The config specifies:
- **Which fields** to include (`fields[]` with `path` and `from dot-path`)
 - **Array indexing** (`emails[0]` → first email as string)
 - **List mapping** (`skills[].name` → flat list of skill names)
 - **Whether to normalize** each field (per-field and global toggle)
 - **How to handle missing values** (`null / omit / error`)
 - **Whether to include metadata** (`include_confidence, include_provenance`)

The projection engine reads from the canonical record and emits a reshaped dict. This is validated against a JSON schema dynamically generated from the config.

Edge Cases (10 handled, 116 passing tests)

Scenario	Handling
Source file missing / garbage JSON	Skip, log, continue — pipeline never crashes
Same candidate, different name spellings	Merged via email match; highest-priority name wins
Conflicting field values across sources	Source priority picks winner; confidence -0.15
Phone without country code	Default US via phonenumbers; null if still invalid
Unicode/international names	Preserved exactly — no data loss
Skills from multiple sources	Union merge (additive), not pick-winner
Same phone, different formats	Both normalize to E.164, then matched

Scope-Outs

1. **LinkedIn scraping** — mocked as JSON fixture (ToS). Production: LinkedIn Partner API.
2. **ML skill extraction** — using alias-map + fuzzy match. Production: fine-tuned NER.
3. **Horizontal scaling** — in-memory for thousands. For millions: shard by email hash, Spark.